FICHE MÉMO

Piloter le développement dans la chaîne de production

Les notions du Module 3, dans l'ordre du deck : on quitte le poste du développeur pour piloter la chaîne de production collective, du Context Nexus à la gouvernance d'entreprise, en passant par la CI, la CD et l'observabilité.

1. Du dev solo au pilotage du SDLC d'entreprise

Au Module 2, l'IA était un assistant personnel sur votre machine. Au Module 3, elle entre dans la **chaîne de production collective**.

Périmètre	Module 2 : sur le poste	Module 3 : dans la chaîne
Posture	Développeur assisté (code, refactor)	Pilote du cycle de production (SDLC)
Artefact	Prompt local, agent individuel	Context Nexus & IDP d'entreprise
Gouvernance	Auto-contrôle, tests locaux	Quality Gates CI/CD, Evals & Observabilité
Objectif	Efficacité individuelle ou en binôme	Performance & alignement d'équipe

> On n'écrit plus de code à la main : on **pilote** le SDLC augmenté (Software Development Life Cycle étendu aux équipes produit).

La vélocité n'est pas la valeur (étude du MIT, Mert Demirer et al., 2025/2026) : l'IA est un **amplificateur**, elle accélère les bonnes comme les mauvaises décisions. Produire 2x plus vite une fonctionnalité mal spécifiée détruit de la valeur ; sans cadre d'ingénierie, on produit de l'**AI Slop** (dette technique, code incompréhensible). La valeur se déplace vers la **qualité des décisions qui précèdent le code** et la **rigueur des contrôles (gates)**.

Évolution des rôles : du PO au duo **Product Manager + UX Designer** (le découpage de tickets s'automatise avec le Spec-Driven Development) ; la **squad râteau** (OCTO) : le manche = l'expertise principale (Dev, Archi, QA), les dents = les compétences secondaires décuplées par les agents.

Calibrer l'équipe selon le Core Domain Chart (DDD) : c'est la **charge cognitive métier** qui fixe la taille de l'équipe, pas le volume de code.

Sous-domaine	Impact stratégique	Équipe recommandée
Core Domain	Différenciation forte, métier complexe	4 séniors (Archi, Dev, QA, DevSecOps), revue humaine stricte
Supporting Domain	Complexité moyenne	2 séniors + 1 junior, automatisation encadrée

Sous-domaine	Impact stratégique	Équipe recommandée
Generic Domain	Métier standard / CRUD	1 sénior + 1 junior, automatisation très poussée par agents

Le SDLC augmenté en 5 moments continus : une boucle de transformation de la connaissance.

1. **Découvrir** : Product Discovery & EventStorming.
2. **Concevoir** : DDD, ADR & Context Factory.
3. **Construire** : Spec-Driven Dev & Worktrees.
4. **Valider / Déployer** : revue PR par agent, Sonar & Gates.
5. **Apprendre** : observabilité d'équipe & FinOps.

> Principe directeur : comprendre → expliciter → concevoir → exécuter → vérifier → apprendre.

2. Le Context Nexus & l'Internal Developer Platform (IDP)

L'entropie de la connaissance : à chaque transmission, la connaissance métier se perd (besoin métier → story résumée, perte 20 % → interprétation dev, perte 40 % → dérive finale du code). Les développeurs compensaient par leur intuition ; les agents n'ont pas d'intuition métier : **toute ambiguïté ou règle implicite devient un bug généré**.

Le Context Nexus centralise l'intention métier de façon explicite, structurée et adaptée à chaque profil. À l'échelle d'une entreprise, l'**IDP** (Backstage, Port) devient ce Context Nexus, exposé via MCP / OpenTelemetry : la passerelle contextuelle consultée par les humains et les agents.

Niveau d'autorité	Contenu	Rédigé par	Statut pour l'IA
1. Invariants d'entreprise	Normes SecOps, RGPD, Archi DSI	La Gouvernance	Immuables (lecture seule)
2. Invariants métier & sachants	Agrégats DDD, règles métier, ADR	Les sachants (PM/BA/Archi)	Arbitrage humain synchrone
3. Documentation opérationnelle dérivée	OpenAPI, schemas, changelogs	Les agents IA en CI/CD	Générée et maintenue à jour

> L'IA ne peut jamais modifier un invariant SecOps ou réécrire une règle métier sans validation explicite des sachants.

Les profils : PM & BA (intention fonctionnelle, critères d'acceptation, scénarios BDD), Tech Lead & Architecte (ADR, frontières DDD), SecOps & Gouvernance (règles de sécurité et de conformité), Développeur & Agent IA (le développeur pilote l'agent, qui interroge l'IDP via MCP).

Bi-directional Sync : l'agent bloque la CI si le code enfreint une règle du sachant, et génère une PR de documentation dérivée si le code fait évoluer une API.

3. IA dans la CI : revue de PR & SonarQube

Dimension	Module 2 : revue locale	Module 3 : revue PR en CI
Exécution	Hook pré-commit local	Runner CI à chaque Pull Request
Périmètre vu	Diff stagé local + conventions	PR complète + Spec + Rules + SonarQube
Restitution	Prompt local individuel	Commentaire de PR + Quality Gate

```bash
# Bonne pratique : fournir un diff riche en contexte à l'agent CI
git diff -U10 origin/main...HEAD
```

> Le `-U10` (10 lignes de contexte autour du changement) divise par 3 les faux positifs par rapport au `-U3` par défaut.

Analyse augmentée SonarQube : un agent CI de tri entre le rapport brut et la PR. **Filtrage intelligent** des avertissements non pertinents, **explication pédagogique** de la faille (OWASP) et de son risque réel, **remédiation immédiate** avec un patch prêt à appliquer.

Pair-programming à trois (pratique Zenika) : au lieu du cycle asynchrone (PR, attente, commentaires, attente, correction), **Dev + Relecteur + Agent en synchrone** : l'agent de CI produit son rapport, le binôme valide les remarques en direct, l'agent applique les corrections immédiatement.

4. IA dans la CD : Deployment Gates & Release

Deployment Gates : Build & Evals (CI validée) → Deployment Gate IA (non-drift & specs) → mise en prod / staging K8s. En cas d'échec ou de faille de spec : interruption & alerte dev.

- **Evals de non-dérive** : rétro-compatibilité des contrats d'API et des schémas de données.
- **Critères d'acceptation** : l'agent vérifie que tous les scénarios de la spec sont couverts.

Politique de merge graduée (Human-in-the-Loop) :

Type de PR	Politique de merge
PR mineure / doc / fix mineur	Auto-merge encadré par Evals CI
PR fonctionnelle / métier	Revue synchrone Dev + Relecteur
PR archi / Core Domain / sécurité	Double validation humaine obligatoire

> Règle absolue : sur le **Core Domain**, aucun merge automatique sans validation humaine. Les agents préparent les preuves (evals, diffs, specs) pour accélérer la décision humaine.

Release orchestration : release notes croisées (technique pour les devops, fonctionnelle pour les PM) et mise à jour du ticketing (ticket Jira / GitLab passé en *In QA* ou *Deployed*, résumé de l'implémentation réelle attaché).

5. Observabilité d'équipe, FinOps & coûts induits

Deux observabilités : l'**applicative** (santé de l'application en production : Prometheus, Grafana, logs, métriques) et celle de l'**usage de l'IA** (tokens, coûts par projet, sous-agents, latences), sujet de Tech Lead et de CTO. AI Gateway (LiteLLM / Portkey) → export OpenTelemetry → dashboard d'équipe (Grafana / Datadog).

L'AI Gateway d'entreprise :

- **point d'agrégation unique** : aucune clé API en libre-service sur les postes ;
- **télémétrie OpenTelemetry** : tokens entrés/sortis, modèles sollicités, coût au centime près ;
- **détection des runaway agents** : arrêt automatique des sous-agents coincés dans des boucles.

> Histoires vraies : 6 sous-agents non supervisés consommant 1 million de tokens en 30 minutes ; un projet interne à 4 500 €/mois faute de gestion des context windows.

Le budget caché : l'adoption agentique multiplie les PRs, donc la charge CI. Budget A = tokens LLM, budget B = infrastructure CI/CD (compute, runners, parallélisme). Un budget IA = **tokens + infrastructure CI + stockage + temps de revue**.

6. Gouvernance d'entreprise, registres MCP & capitalisation

- **Registres de skills d'entreprise** (`skill.sh` , Backstage) : référentiel centralisé, audité et validé par la DSI ; une skill proposée par une équipe passe un audit de sécurité et de qualité avant diffusion.
- **Audit de qualité** des skills : structure respectée, pas de prompts vulnérables, maintenance.
- **Agent archiviste & mémoire d'équipe** : les agents apprennent des sessions et décisions précédentes.

MCP en entreprise :

- **contrôle des transports** : pas d'exécutables locaux non vérifiés, conteneurs isolés (Docker) ou passerelles HTTP sécurisées ;
- **habilitations d'écriture** : lecture sur le ticketing (Jira, Linear), écriture sur le backlog soumise à validation explicite ;
- **registres MCP** : un registre peut bloquer un serveur MCP officiel qui ne respecte pas la politique de transport.

Réversibilité et souveraineté : dissocier le **harness** (l'outil) du fournisseur de modèles (LLM) ; préserver le choix des modèles via des passerelles (basculer de A à B sans réécrire l'outillage) ; résidence des données et conformité (aucune donnée d'entreprise pour le réentraînement).

Les quatre démonstrations

Démo	Scénario
1. Context Nexus & IDP	Consulter le Context Nexus dans un portail développeur ; définir un invariant SecOps (lecture seule) et une spec métier du PM ; soumettre une PR : l'agent interroge l'IDP via MCP, vérifie les normes SecOps et détecte un écart métier ; générer la documentation dérivée (OpenAPI) sans altérer la spec métier.

Démo	Scénario
2. Revue PR agentique & SonarQube	Une PR avec une modification d'API et une faille d'injection cachée ; le pipeline CI interroge l'API SonarQube et lit le diff `-U10` ; rapport synthétique sur la PR avec correctif ; validation en binôme et application du patch.
3. Deployment Gate & Release	Valider une release candidate ; le Deployment Gate vérifie critères d'acceptation et API ; déployer sur Kubernetes / staging ; générer la release note et mettre à jour le ticket.
4. Observabilité & FinOps	Dashboard Grafana alimenté par l'OpenTelemetry d'une AI Gateway ; repérer un pic causé par un sous-agent de revue mal configuré ; poser limites de budget et règles de découpage de contexte ; simuler l'alerte et bloquer les requêtes hors budget.

La suite

Établir la feuille de route d'adoption du SDLC augmenté dans votre organisation.